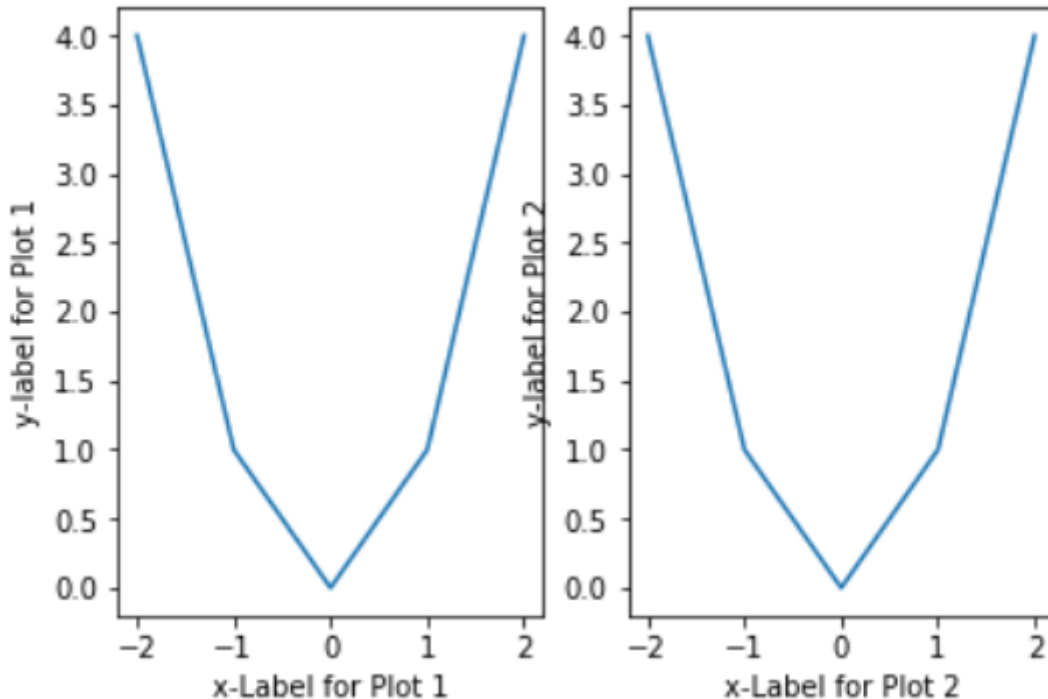


Линейные графики(продолжение)

Иногда, когда мы объединяем несколько графиков, некоторые элементы могут перекрываться и сделать рисунок нечитаемым:



Мы можем настроить интервал между нашими графиками, чтобы убедиться, что созданная нами фигура видна и проста для понимания. Для этого мы используем команду `plt.subplots_adjust()`. `plt.subplots_adjust()` имеет несколько аргументов ключевого слова, которые могут перемещать ваши графики внутри рисунка:

- `left` - левое поле, по умолчанию 0,125. Вы можете увеличить это число, чтобы освободить место для метки оси Y
- `right` - правое поле, по умолчанию 0,9. Вы можете увеличить его, чтобы освободить место для фигуры, или уменьшить, чтобы освободить место для легенды.
- `bottom` - нижнее поле, по умолчанию 0,1. Вы можете увеличить это значение, чтобы освободить место для меток галочки или метки оси X
- `top` - верхнее поле, по умолчанию 0,9
- `wspace` - горизонтальное пространство между соседними подзаголовками, по умолчанию 0,2
- `hspace` - вертикальное пространство между смежными подзаголовками, по умолчанию 0,2

Например, если бы мы добавляли пробел в нижнюю часть графика, изменяя нижнее поле на 0,2 (вместо значения по умолчанию 0,1), мы бы использовали команду:

```
plt.subplots_adjust(bottom=0.2)
```

Мы также можем использовать несколько аргументов ключевого слова, если нам нужно настроить несколько полей. Например, мы можем настроить как верхнюю часть, так и пространство `hspace`:

```
plt.subplots_adjust(top=0.95, hspace=0.25)
```

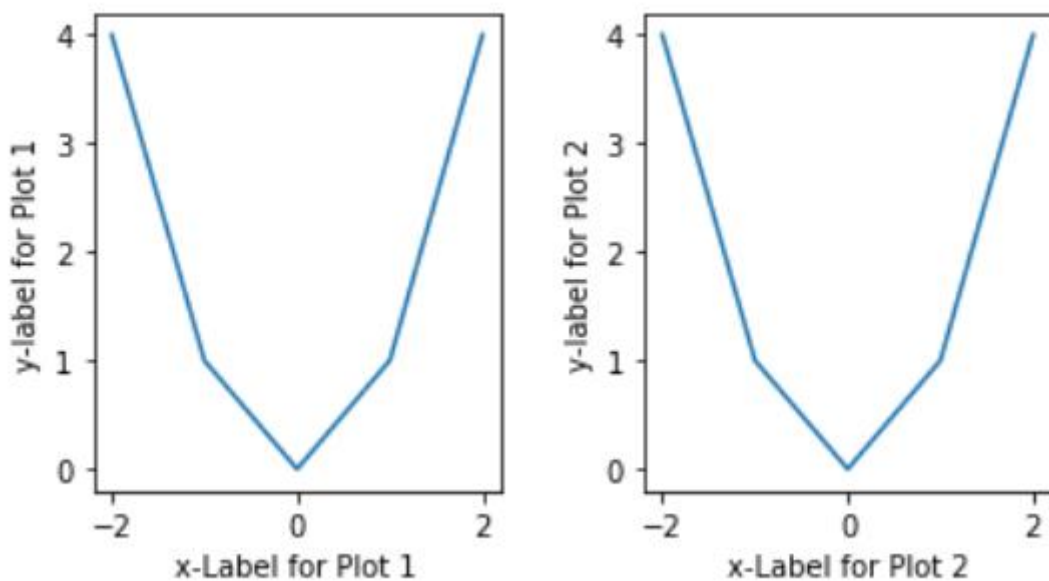
Давайте воспользуемся `wspace`, чтобы исправить рисунок выше:

```
# Left Plot
plt.subplot(1, 2, 1)
plt.plot([-2, -1, 0, 1, 2], [4, 1, 0, 1, 4])

# Right Plot
plt.subplot(1, 2, 2)
plt.plot([-2, -1, 0, 1, 2], [4, 1, 0, 1, 4])

# Subplot Adjust
plt.subplots_adjust(wspace=0.35)

plt.show()
```

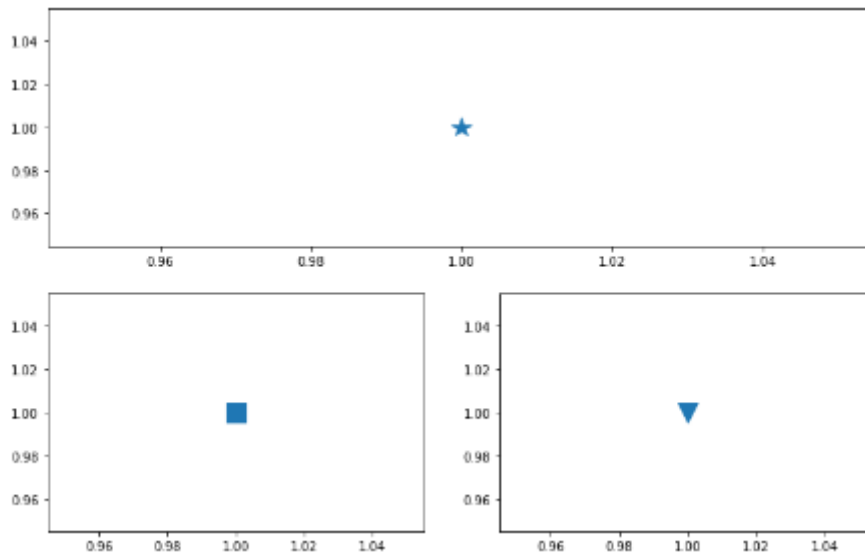


Задание

1. Мы собираемся создать фигуру, состоящую из двух рядов графиков. В нем должны быть:

один график в верхнем ряду

два графика в нижнем ряду



```
from matplotlib import pyplot as plt

x = range(7)
straight_line = [0, 1, 2, 3, 4, 5, 6]
parabola = [0, 1, 4, 9, 16, 25, 36]
cubic = [0, 1, 8, 27, 64, 125, 216]
```

Начните с использования метода `subplot` для создания экземпляра `subplot` в верхней строке (прямоугольник со звездой в нем).

2. Постройте зависимость `straight_line` и `x` на выбранной вами подзаголовке.
3. Теперь используйте команду `plt.subplot()`, чтобы выбрать прямоугольник в первом столбце второй строки (тот, в котором квадрат на рисунке выше). Постройте график зависимости `parabola` от `x` в этом поле.
4. Теперь используйте команду `plt.subplot()`, чтобы выбрать поле во втором столбце второй строки (тот, в котором есть треугольник). Постройте зависимость `cubic` и `x` в этом поле.
5. Увеличьте интервал между горизонтальными частями графика до 0,35 и нижнее поле до 0,2.

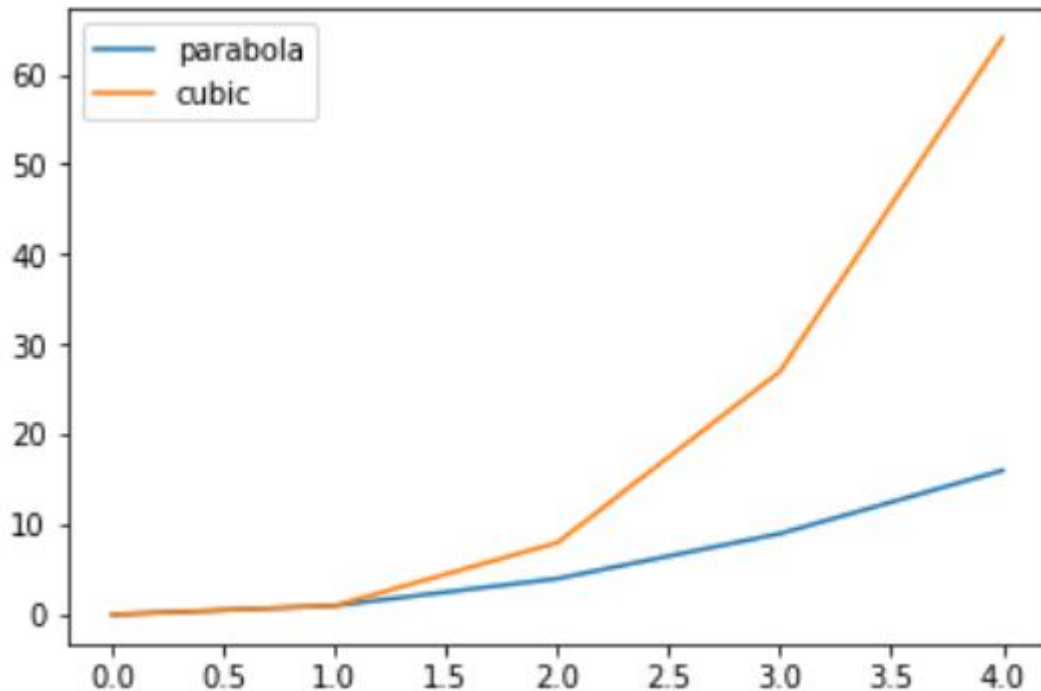
Легенда

Когда у нас есть несколько линий на одном графике, мы можем пометить их с помощью команды `plt.legend()`.

Метод легенды принимает список с отображаемыми метками. Так, например, мы можем вызвать:

```
plt.plot([0, 1, 2, 3, 4], [0, 1, 4, 9, 16])
plt.plot([0, 1, 2, 3, 4], [0, 1, 8, 27, 64])
plt.legend(['parabola', 'cubic'])
plt.show()
```

который отобразит легенду на нашем графике, пометив каждую строку:



`plt.legend()` также может принимать аргумент ключевого слова `loc`, который позиционирует легенду на рисунке.

`Loc` принимает следующие значения параметров:

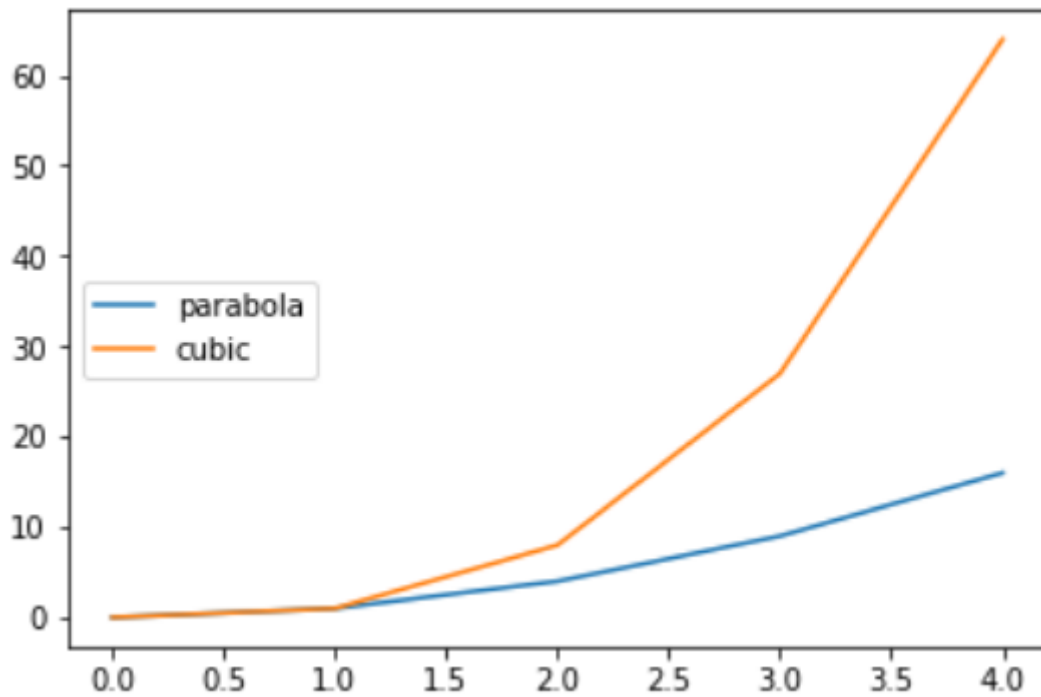
Number	Code	String
0		best
1		upper right
2		upper left
3		lower left
4		lower right
5		right
6		center left
7		center right
8		lower center
9		upper center
10		center

Примечание. Если вы решите не устанавливать значение для `loc`, по умолчанию будет выбрано «лучшее» местоположение.

Например, мы можем вызвать `plt.legend()` и установить для `loc` значение 6:

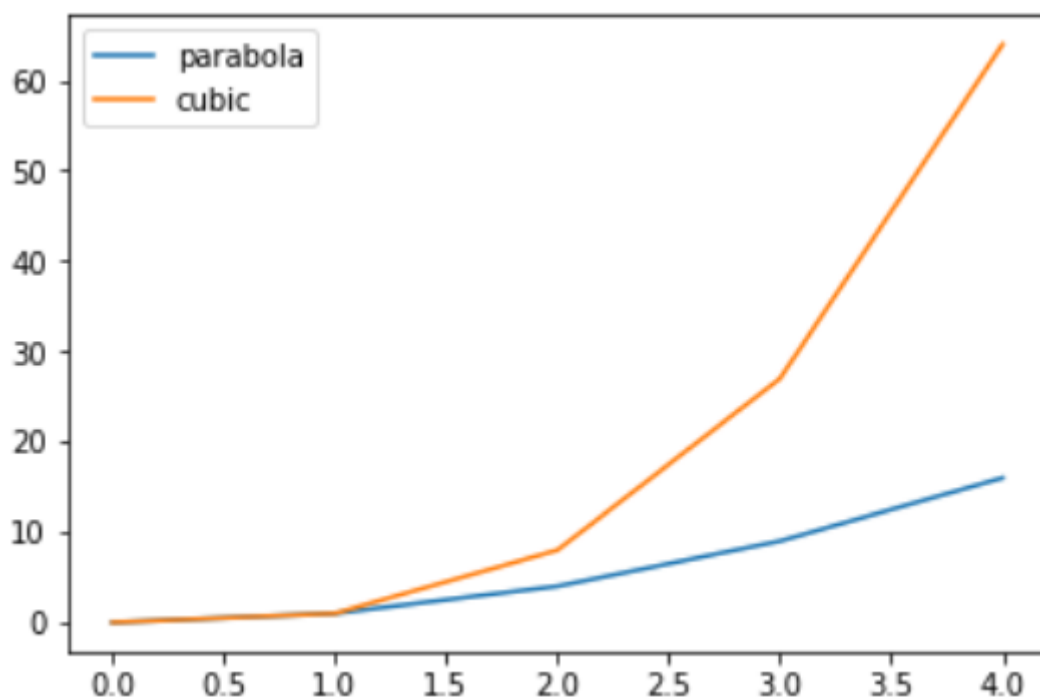
```
plt.legend(['parabola', 'cubic'], loc=6)
plt.show()
```

что переместит легенду в левую часть графика:



Иногда проще пометить каждую строку по мере ее создания. Если мы хотим, мы можем использовать метку ключевого слова внутри `plt.plot()`. Если мы выберем это, мы не передадим никаких меток в `plt.legend()`. Например:

```
plt.plot([0, 1, 2, 3, 4], [0, 1, 4, 9, 16],  
         label="parabola")  
plt.plot([0, 1, 2, 3, 4], [0, 1, 8, 27, 64],  
         label="cubic")  
plt.legend() # Still need this command!  
plt.show()
```



Задание

1. Мы нарисовали для вас три линии. Они отображают температуру за последний год в трех местах:

- Хайрул (hyrule)
- Какарико (какарико)
- Долина герудо (герудо)

Создайте список строк, содержащий «Hyrule», «Kakariko» и «Gerudo Valley», и сохраните его в переменной с именем `legend_labels`.

```
from matplotlib import pyplot as plt

months = range(12)
hyrule = [63, 65, 68, 70, 72, 72, 73, 74, 71, 70, 68, 64]
kakariko = [52, 52, 53, 68, 73, 74, 74, 76, 71, 62, 58, 54]
gerudo = [98, 99, 99, 100, 99, 100, 98, 101, 101, 97, 98, 99]

plt.plot(months, hyrule)
plt.plot(months, kakariko)
plt.plot(months, gerudo)
```

2. Создайте легенду для графика, введя `legend_labels` в `plt.legend()`.
3. Установите легенду в центре диаграммы.

Изменить метки

Во всех наших предыдущих упражнениях наши команды начинались с `plt` .. Чтобы изменить метки на осях, нам нужно попробовать что-то немного другое.

Поскольку наши графики могут иметь несколько подграфиков, мы должны указать, какой из них мы хотим изменить. Для этого мы вызываем `plt.subplot()` по-другому.

```
ax = plt.subplot(1, 1, 1)
```

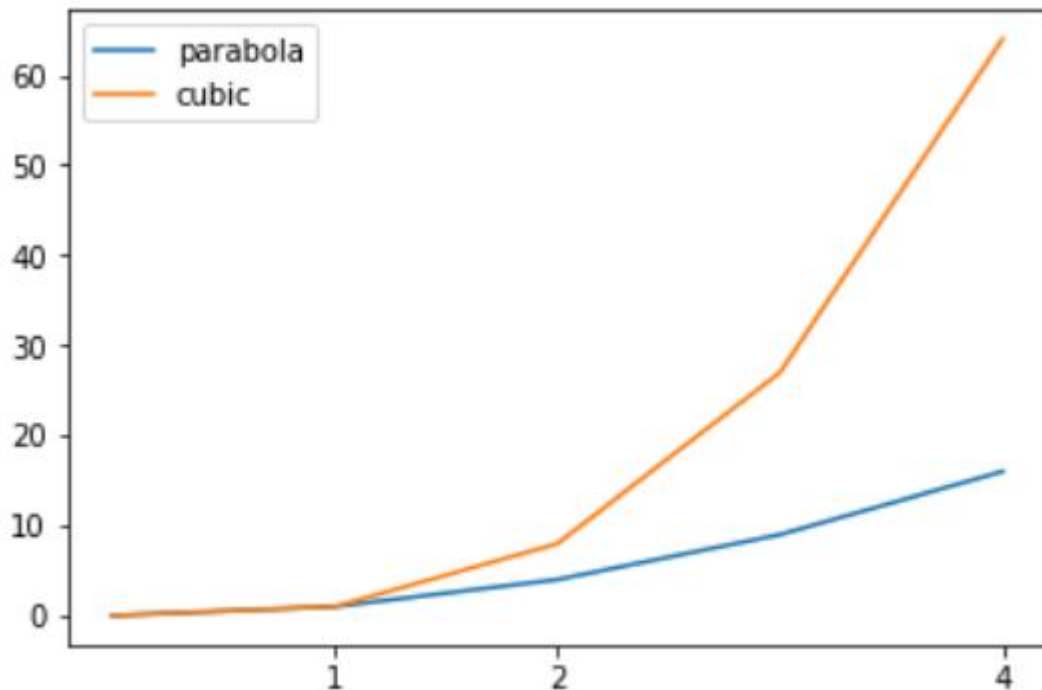
`Ахе` — это объект оси, и он позволяет нам изменять оси, принадлежащие определенному подзаголовку. Даже если у нас есть только один график, когда мы хотим изменить тики, нам нужно будет начать с вызова либо `ax = plt.subplot(1, 1, 1)`, либо `ax = plt.subplot()`, чтобы получить наши оси.

Предположим, мы хотим установить для наших x-меток 1, 2 и 4. Мы будем использовать следующий код:

```
ax = plt.subplot()
plt.plot([0, 1, 2, 3, 4], [0, 1, 4, 9, 16])
```

```
plt.plot([0, 1, 2, 3, 4], [0, 1, 8, 27, 64])  
ax.set_xticks([1, 2, 4])
```

Наш результат будет выглядеть так:

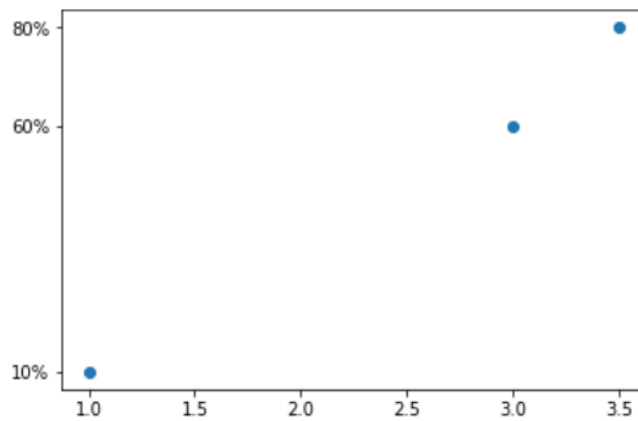


Мы также можем изменить у-метки с помощью `ax.set_yticks()`.

Если нам нужны специальные метки (например, строки), мы можем использовать команду `ax.set_xticklabels()` или `ax.set_yticklabels()`. Например, нам может потребоваться ось Y с отметками 0,1, 0,6 и 0,8, но обозначить их 10%, 60% и 80% соответственно. Для этого мы используем следующие команды:

```
ax = plt.subplot()  
plt.plot([1, 3, 3.5], [0.1, 0.6, 0.8], 'o')  
ax.set_yticks([0.1, 0.6, 0.8])  
ax.set_yticklabels(['10%', '60%', '80%'])
```

Это приведет к такой маркировке оси Y:



1. Представим, что мы работаем на компанию Dinnersaur, которая доставляет обеды людям, которые не хотят готовить. Dinnersaur недавно запустил новую услугу, основанную на подписке, где пользователи подписываются на питание в течение всего года вместо того, чтобы покупать блюда по запросу.

Мы нарисовали для вас линейный график, представляющий долю пользователей, которые перешли на эту новую услугу с момента ее развертывания в январе. Сначала сохраните набор осей в переменной с именем `ax`. Мы будем использовать `ax`, чтобы установить отметки `x` и `y` и метки, чтобы сделать этот график более удобным для чтения.

```
from matplotlib import pyplot as plt

month_names = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

months = range(12)
conversion = [0.05, 0.08, 0.18, 0.28, 0.4, 0.66, 0.74, 0.78, 0.8, 0.81, 0.85, 0.85]

plt.xlabel("Months")
plt.ylabel("Conversion")

plt.plot(months, conversion)

# Your work here

plt.show()
```

2. Используя `Axe`, установите `x`-метки, чтобы они соответствовали списку месяцев.
3. Установите метки `x-tick` в качестве списка `month_names`.
4. Обозначьте оси `y` как проценты, соответствующие значениям `[0,10, 0,25, 0,5, 0,75]`, а не десятичные дроби.

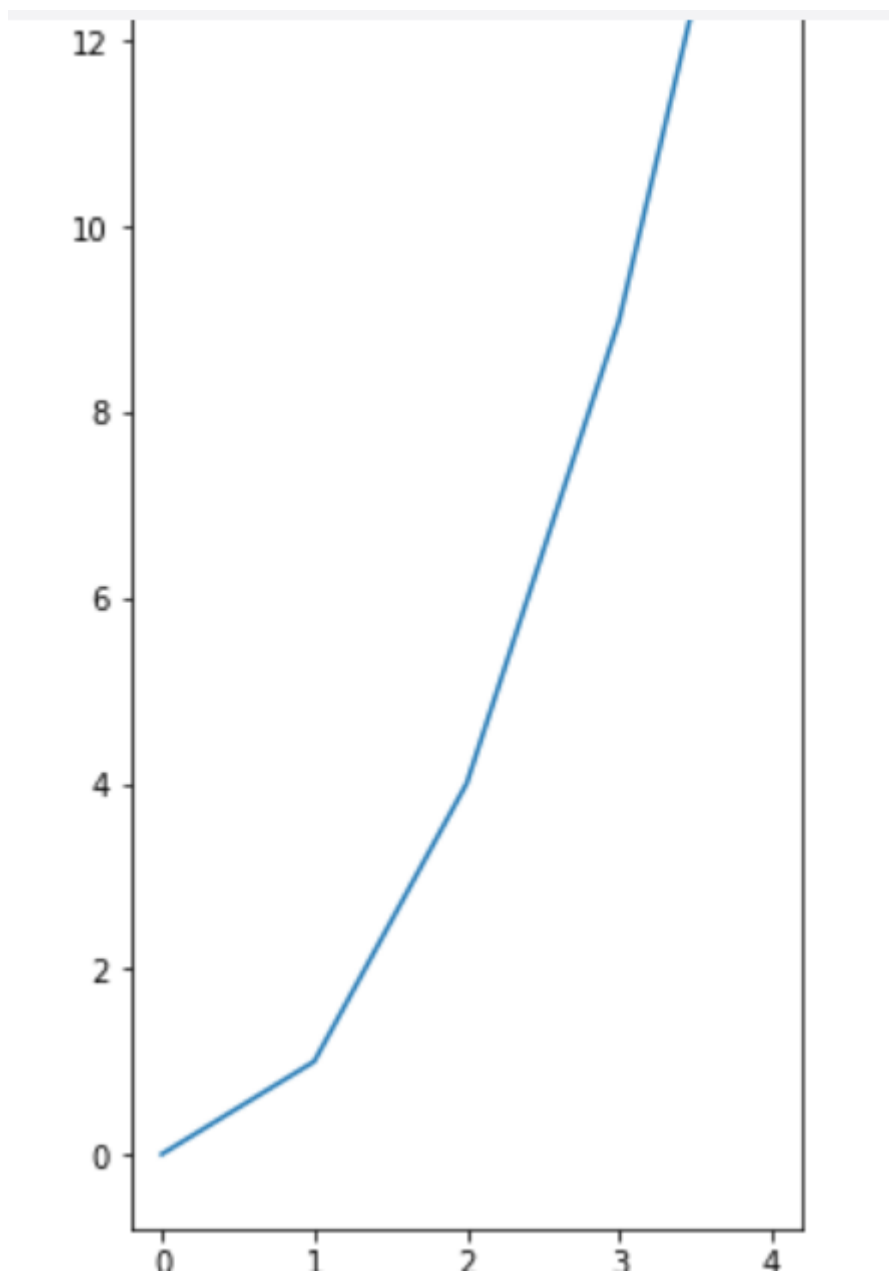
Фигуры

Когда мы строим много графиков, легко получить линии, которые были нанесены, но не отображены. Если мы не будем осторожны, эти «забытые» линии появятся на ваших новых графиках. Чтобы быть уверенным, что у вас нет случайных линий, вы можете использовать команду `plt.close('all')`, чтобы очистить все существующие графики перед построением нового.

Ранее мы узнали, как поместить два набора осей на одну фигуру. Иногда мы предпочли бы иметь две отдельные фигуры. Мы можем использовать команду `plt.figure()` для создания новых фигур и их размера, как мы хотим. Мы можем добавить ключевое слово `figsize = (width, height)`, чтобы установить размер фигуры в дюймах. Мы используем круглые скобки ((и)) для передачи ширины и высоты, которые разделяются запятой (,).

```
plt.figure(figsize=(4, 10))
```

Он выглядел бы высоким и вытянутым, вот так:



Создав рисунок, мы можем сохранить его, чтобы использовать в презентации или на веб-сайте. Мы можем использовать команду `plt.savefig()` для сохранения во многих различных форматах файлов, таких как `png`, `svg` или `pdf`. После построения мы можем вызвать `plt.savefig('name_of_graph.png')`:

```
# Figure 2
plt.figure(figsize=(4, 10))
plt.plot(x, parabola)
plt.savefig('tall_and_narrow.png')
```

Это сохранит файл `high and thin.png` в вашей файловой системе.

Задание

1. Во-первых, закройте все графики, чтобы убедиться, что у нас нет уже нанесенных линий, о которых мы забыли.

```
from matplotlib import pyplot as plt

word_length = [8, 11, 12, 11, 13, 12, 9, 9, 7, 9]
power_generated = [753.9, 768.8, 780.1, 763.7, 788.5, 782, 787.2, 806.4, 806.2, 798.9]
years = [2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009]
```

2. Создайте фигуру и нанесите на график `word_length` против лет. Этот набор данных представляет собой длину слов, победивших в конкурсе Scripps National Spelling Bee за 11 лет. Сохраните эту фигуру в файле под названием «`win_word_lengths.png`».
3. На следующей строке создайте фигуру шириной 7 дюймов и высотой 3 дюйма и постройте график `power_generated` в зависимости от лет. Этот набор данных представляет собой мощность, вырабатываемую атомными станциями в США за 11 лет. Сохраните этот рисунок в файле под названием `power_generated.png`.

Заключение

Теперь вы попрактиковались с несколькими двумерными линейными графиками в Matplotlib. Вы видели, как можно создавать простые, читаемые графики с помощью нескольких команд. Вы также узнали несколько команд, позволяющих лучше стилизовать и маркировать ваши сюжеты. Вот концепции, которые вы уже видели в Matplotlib:

- Создание линейного графика из данных

- Изменение внешнего вида линии
- Увеличение разных частей оси
- Нанесение меток на заголовки и оси
- Создание более сложного макета фигуры
- Добавление легенд к графикам
- Изменение меток и позиций галочки
- Сохранение того, что вы сделали

Двигаясь дальше, мы узнаем, как создавать различные типы графиков (помимо линейных графиков!) В Matplotlib и как выбирать между этими графиками при отображении данных.

Задание

1. Определите три списка, x , y_1 и y_2 , и заполните их целыми числами. Эти числа могут быть чем угодно, но было бы неплохо, если бы они были фактическими показателями, которые вы хотите сравнивать. Данные можно посмотреть на этом сайте: <http://www.tylervigen.com/spurious-correlations>
2. Постройте график зависимости y_1 и x и отобразите график.
3. На том же графике постройте y_2 vs x (после линии, где вы строите y_1 vs x)
4. Сделайте линию y_1 розовой линией, а линию y_2 серой линией. Дайте обеим линиям круглые маркеры.
5. Дайте своему графику название «Две линии на одном графике», а ось X - «Великолепная ось X », а ось Y - «Потрясающая ось Y ».
6. Создайте для графика легенду и поместите ее в правом нижнем углу.
7. Попробуйте применить различные методы, изученные в данном разделе. Может быть, сделайте какие-нибудь подграфики и разделите линии. Потренируйтесь с увеличением определенных частей графика или выбором определенных x - или y -отметок для отображения.